

Practical - 2

AIM: Hadoop Installation, Configuration and HDFS Operations on Windows/Linux

PROBLEM DEFINITION:

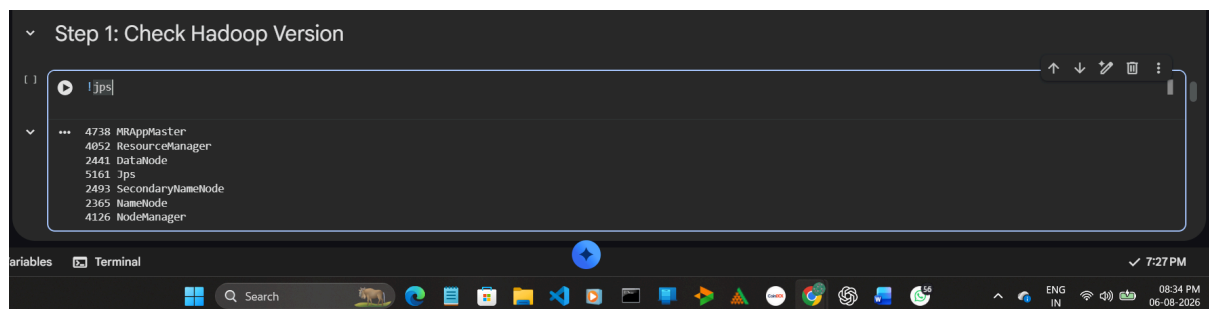
You have recently joined DataAnalytics Inc. as a Hadoop Administrator. The organization processes large-scale customer, sales, and marketing data using the Hadoop ecosystem. Due to increasing data volumes, the company has decided to migrate its data storage infrastructure from traditional file systems to Hadoop Distributed File System (HDFS).

As a Hadoop Administrator, your responsibility is to install and configure Hadoop in a Single-Node Cluster environment on Windows/Linux, verify the Hadoop services, manage HDFS storage, and perform data management operations.

Management wants a fully functional Hadoop environment that can store, retrieve, and process big data efficiently. Your task is to install Hadoop, configure the Hadoop ecosystem, start Hadoop services, and perform HDFS operations using Hadoop commands.

OUTPUT :

Step 1: Check Hadoop Version



The screenshot shows a Windows Terminal window titled "Step 1: Check Hadoop Version". The command prompt shows the command `jps` has been executed. The output lists several Java processes running on the system:

```
4738 MRAppMaster
4052 ResourceManager
2441 DataNode
5161 Jps
2493 SecondaryNameNode
2365 NameNode
4126 NodeManager
```

The terminal window includes a taskbar at the bottom with various application icons and a system tray showing the time as 7:27 PM and date as 06-08-2026.

Step 2: Access Hadoop Web UI

```
Step 2: Access Hadoop Web UI

[ ] lhadoop version

Hadoop 3.4.1
Source code repository https://github.com/apache/hadoop.git -r 4d7825309348956336b8f06a08322b78422849b1
Compiled by mthakur on 2024-10-09T14:57Z
Compiled on platform linux-x86_64
Compiled with protoc 3.23.4
From source with checksum 7292fe9dba5e2e44e3a9f763fce3e680
This command was run using /content/hadoop-3.4.1/share/hadoop/common/hadoop-common-3.4.1.jar
```

Step 3: Create Directories in HDFS

```
Step 3: Create Directories in HDFS

[ ] lhdfs dfs -mkdir /company
lhdfs dfs -mkdir /company/customer
lhdfs dfs -mkdir /company/sales
lhdfs dfs -mkdir /company/marketing
```

Step 4: Create Sample Files

```
Step 4: Create Sample Files

[ ] %writefile customer.txt
101,Rahul
102,Priya
103,Amit

Writing customer.txt

[ ] %writefile sales.txt
101,5000
102,7000
103,8000

Writing sales.txt
```

Step 5: Upload Files to HDFS

```
Step 5: Upload Files to HDFS

[ ] lhdfs dfs -put customer.txt /company/customer/

[ ] lhdfs dfs -put sales.txt /company/sales/

[ ] lhdfs dfs -ls /company/customer

Found 1 items
-rw-r--r-- 1 root supergroup 29 2026-08-06 13:50 /company/customer/customer.txt
```

Step 6: List HDFS Files

```
[ ] lhdfs dfs -ls /  
Found 5 items  
drwxr-xr-x - root supergroup 0 2026-08-06 13:50 /company  
drwxr-xr-x - root supergroup 0 2026-08-06 13:50 /output  
drwxr-xr-x - root supergroup 0 2026-08-06 13:49 /test  
drwxr-xr-x - root supergroup 0 2026-08-06 13:50 /tmp  
drwxr-xr-x - root supergroup 0 2026-08-06 13:49 /user  
[ ] lhdfs dfs -ls /company  
Found 3 items  
drwxr-xr-x - root supergroup 0 2026-08-06 13:50 /company/customer  
drwxr-xr-x - root supergroup 0 2026-08-06 13:50 /company/marketing  
drwxr-xr-x - root supergroup 0 2026-08-06 13:50 /company/sales  
[ ] lhdfs dfs -ls -R /
```

Step 7: Display File Contents

```
Step 7: Display File Contents  
lhdfs dfs -cat /company/customer/customer.txt  
101,Rahul  
102,Priya  
103,Amit  
lhdfs dfs -cat /company/sales/sales.txt  
101,5000  
102,7000  
103,8000
```

Step 8: Copy Files

```
Step 8: Copy Files  
[ ] lhdfs dfs -cp /company/customer/customer.txt /company/customer/customer_copy.txt  
[ ] lhdfs dfs -ls /company/customer  
Found 2 items  
-rw-r--r-- 1 root supergroup 29 2026-08-06 13:50 /company/customer/customer.txt  
-rw-r--r-- 1 root supergroup 29 2026-08-06 13:51 /company/customer/customer_copy.txt  
Step 9: Move File
```

Step 9: Move File

```
Step 9: Move File  
[ ] lhdfs dfs -mv /company/customer/customer_copy.txt /company/marketing/  
[ ] lhdfs dfs -ls /company/marketing  
Found 1 items  
-rw-r--r-- 1 root supergroup 29 2026-08-06 13:51 /company/marketing/customer_copy.txt
```

Step 10: Rename File

```
Step 10: Rename File

[ ] 1) hdfs dfs -mv /company/marketing/customer_copy.txt /company/marketing/customer_backup.txt

[ ] 2) hdfs dfs -ls /company/marketing

Found 1 items
-rw-r--r-- 1 root supergroup      29 2026-08-06 13:51 /company/marketing/customer_backup.txt

Step 11: Download File
```

Variables Terminal 7:27 PM 08:41 PM 06-08-2026

Step 11: Download File

```
Step 11: Download File

[ ] 1) mkdir downloaded

[ ] 2) hdfs dfs -get /company/marketing/customer_backup.txt downloaded/

[ ] 3) ls downloaded

customer_backup.txt

Step 12: Delete File
```

Variables Terminal 7:27 PM 08:42 PM 06-08-2026

Step 12: Delete File

```
Step 12: Delete File

[ ] 1) hdfs dfs -rm /company/marketing/customer_backup.txt

Deleted /company/marketing/customer_backup.txt

[ ] 2) hdfs dfs -ls /company/marketing

Step 13: Delete Directory
```

Variables Terminal 7:27 PM 08:43 PM 06-08-2026

Step 13: Delete Directory

```
Step 13: Delete Directory

[ ] 1) hdfs dfs -rm -r /company/marketing

Deleted /company/marketing

[ ] 2) hdfs dfs -ls /company

Found 2 items
drwxr-xr-x - root supergroup      0 2026-08-06 13:51 /company/customer
drwxr-xr-x - root supergroup      0 2026-08-06 13:50 /company/sales

Step 14: Check HDFS Report
```

Variables Terminal 7:27 PM 08:43 PM 06-08-2026

Step 14: Check HDFS Report

```
Step 14: Check HDFS Report

l | 1) lhdfs dfsadmin -report

... Configured Capacity: 115658190848 (107.72 GB)
    Present Capacity: 91408585398 (85.13 GB)
    DFS Remaining: 91408228352 (85.13 GB)
    DFS Used: 357046 (348.68 KB)
    DFS Used%: 0.00%
    Replicated Blocks:
        Under replicated blocks: 0
        Blocks with corrupt replicas: 0
        Missing blocks: 0
        Missing blocks (with replication factor 1): 0
        Low redundancy blocks with highest priority to recover: 0
        Pending deletion blocks: 0
    Erasure Coded Block Groups:
        Low redundancy block groups: 0
        Block groups with corrupt internal blocks: 0
        Missing block groups: 0

Variables Terminal 7:27 PM 08:45 PM 06-08-2026
```

Step 15: Verify Block Information

```
Step 15: Verify Block Information

l | 1) lhdfs fsck /company/customer/customer.txt -files -blocks

Connecting to namenode via http://localhost:9870/fsck?ugi=root&files=1&blocks=1&path=%2Fcompany%2Fcustomer%2Fcustomer.txt
FSCK started by root (auth:SIMPLE) from /127.0.0.1 for path /company/customer/customer.txt at Thu Aug 06 13:51:40 UTC 2026

/company/customer/customer.txt 29 bytes, replicated: replication=1, 1 block(s): OK
0. BP-279800794-172.28.0.12-1786024099938:blk_1073741836_1012 len=29 Live_repl=1

Status: HEALTHY
Number of data-nodes: 1
Number of racks: 1
Total dirs: 0
Total symlinks: 0

Replicated Blocks:
Total size: 29 B
Total files: 1
Total blocks (validated): 1 (avg. block size 29 B)

Variables Terminal 7:27 PM 08:45 PM 06-08-2026
```

Step 16: Check Hadoop Cluster Health

```
lhdfs fsck /

... Connecting to namenode via http://localhost:9870/fsck?ugi=root&path=%2F
FSCK started by root (auth:SIMPLE) from /127.0.0.1 for path / at Thu Aug 06 13:51:43 UTC 2026

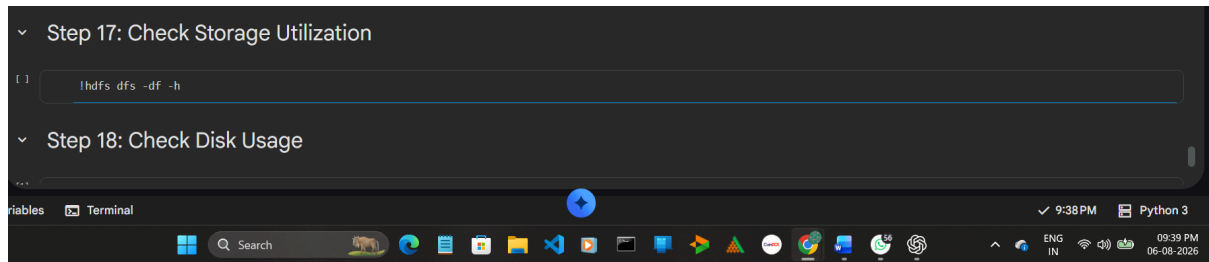
Status: HEALTHY
Number of data-nodes: 1
Number of racks: 1
Total dirs: 17
Total symlinks: 0

Replicated Blocks:
Total size: 329825 B
Total files: 8
Total blocks (validated): 7 (avg. block size 47117 B)
Minimally replicated blocks: 7 (100.0 %)
Over-replicated blocks: 0 (0.0 %)
Under-replicated blocks: 0 (0.0 %)
Mis-replicated blocks: 0 (0.0 %)
Default replication factor: 1
Average block replication: 1.0
Missing blocks: 0
Corrupt blocks: 0
Missing replicas: 0 (0.0 %)
Blocks queued for replication: 0

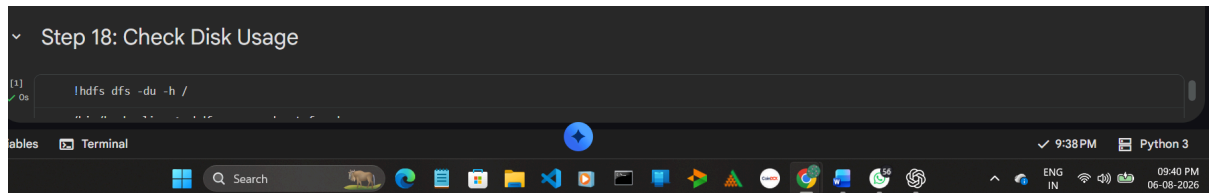
Erasure Coded Block Groups:
Total size: 0 B

Variables Terminal 7:27 PM 08:45 PM 06-08-2026
```

Step 17: Check Storage Utilization



Step 18: Check Disk Usage



KEY QUESTIONS :

Q1. Why is Hadoop preferred for Big Data storage?

Answer:

Hadoop is preferred for Big Data storage because it can store and process very large amounts of data across multiple computers. It provides high scalability, fault tolerance, and cost-effective storage by using commodity hardware. Hadoop automatically replicates data blocks, ensuring that data remains available even if a node fails.

Q2. What is the role of NameNode and DataNode in HDFS?**Answer:**

- NameNode: The NameNode is the master node in HDFS. It manages the file system metadata, such as file names, directory structure, permissions, and the locations of data blocks.
- DataNode: The DataNode is the worker node in HDFS. It stores the actual data blocks, sends block reports to the NameNode, and performs read/write operations as requested by clients.

Q3. What is the difference between HDFS and traditional file systems?

HDFS	Traditional File System
Designed for storing and processing Big Data.	Designed for storing regular files on a single computer.
Stores data across multiple machines.	Stores data on a single machine.
Provides fault tolerance through data replication.	Usually has no built-in replication mechanism.
Highly scalable by adding more nodes.	Scalability is limited by the storage capacity of one system.
Optimized for large files and batch processing.	Suitable for small to medium-sized files and general-purpose use.

Q4. What are the advantages of distributed storage?**Answer:**

The advantages of distributed storage are:

- It can store very large amounts of data across multiple machines.
- It provides fault tolerance by replicating data across different nodes.

- It offers high availability, ensuring data remains accessible even if a node fails.
- It is easily scalable by adding more storage nodes.
- It improves performance by allowing parallel data access and processing.

CONCLUSION :

The Hadoop Single-Node Cluster was successfully installed and configured in Google Colab. Various HDFS operations such as creating directories, uploading, listing, viewing, copying, moving, renaming, downloading, and deleting files were performed successfully. The practical demonstrated how HDFS provides reliable, scalable, and fault-tolerant storage for managing large volumes of data efficiently.

GITHUB : https://github.com/Devsavani12/BIG-DATA_SHP